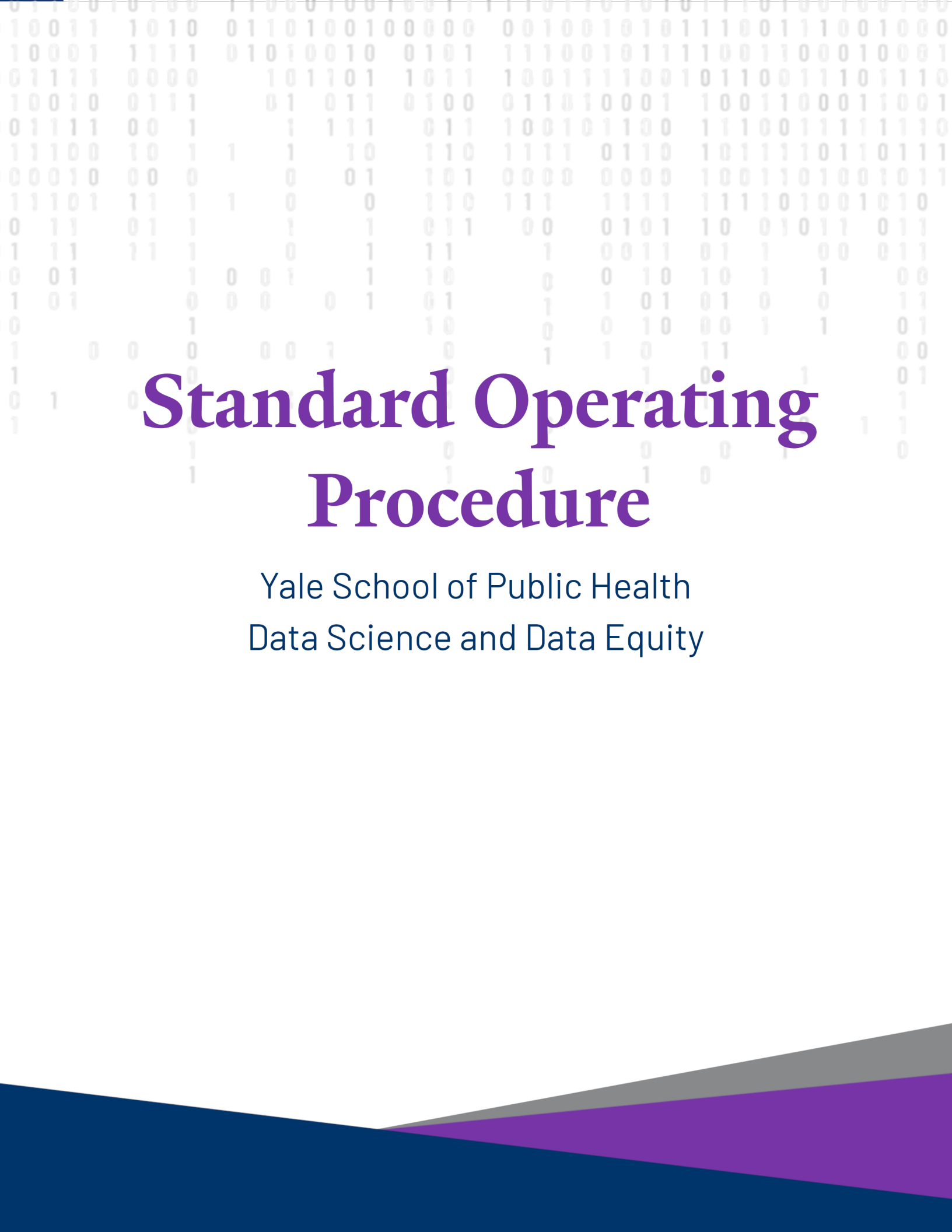




Standard Operating Procedure

Yale School of Public Health
Data Science and Data Equity



The purpose of this document is to outline the contents and ongoing maintenance procedure of the **AI Resources** website.

Developer: Shelby Golden, M.S.

Authors:

- Shelby Golden, M.S.

Date Effective: May 22nd, 2026

Date Updated: May 26th, 2026

Table of contents

Important Links	3
Codebase Overview	3
File Structure	3
Environment Reproducibility	4
Ignored Files	5
Content Curation	6
Policies	6
Courses and Education Programs Curation	6
AI Tools Curation	6
Committee Review of the Curation List	7
Maintaining the Website	7
Feedback and Suggestion Box	7
Index and About Us	9
Policies	9
AI Tools	9
Listing Pages	10
Adding Tiles	10
Removing Tiles	11
Publishing Updates	12
If Rendering Fails to Complete	14

Maintaining the SOP	15
References	16

Important Links

- [Website](#)
- [GitHub Repository](#)
- [SharePoint](#) (located in “DSDE - Core/General/Media & Communications/Website/AI Resources”)

The project SharePoint folder contains additional resources, including running meeting notes, Excel documents tracking the listing queue for approvals and site updates, and links to surveys. This codebase includes local-only files that each user must download and place in their cloned copy of the repository in order to run the website.

Codebase Overview

File Structure

```
ai-resources-page/  
  |- _extensions/  
    |- parse-latex/  
    |- tarleb/  
      |- parse-latex/  
  |- _includes/  
  |- _site/  
  |- _styles/  
    |- Yale_typeface/  
  |- Code/  
    |- Listing_About-Us/  
    |- Listing_Resources/  
    |- Standard Operating Procedure (SOP)/  
      |- SOP Formatting/  
  |- Data/  
  |- Images/  
    |- DSDE Identity/  
    |- People/  
  |- KEEP_LOCAL/  
  |- Pages/  
  |- renv/
```

i Note

Files within the “KEEP LOCAL/Archived Pages/” directory are not documented here, as they are not actively utilized in the CI/CD pipeline. They are maintained for archival purposes only.

Environment Reproducibility

This website was created using R (v 4.5.2) in the RStudio IDE (v 2026.01.1+403) and Quarto (v 1.8.27). The `renv()` package (v 1.1.7) is included to reproduce the same coding environment, ensuring all relevant packages and package versions are stored. If you encounter issues running the scripts, please check that the environment is initialized and that you are using the same versions of R, RStudio, and Quarto.

⚠ Warning

Updating packages, R, or Quarto may introduce compatibility issues with legacy code that will need to be resolved on a case-by-case basis.

If this is the first time the repository is being initialized, you may need to initialize the environment. This step will install all packages and versions used in the codebase, ensuring a reproducible coding environment. To do so:

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/ai-resources-page/"
```

2. Launch the project by opening the *.Rproj file in RStudio.
3. In the R console, activate the environment by running the following lines of code:

Command-Line Interface

```
renv::init()      # Initialize the project.
renv::restore()   # Download packages and their version saved in the
  ↪ lockfile.
```

Note

If you are asked to update packages, say no. The `renv()` is intended to recreate the same environment under which the project was created, making it reproducible. You are ready to proceed when running `renv::restore()` gives the output:

RStudio Console

```
- The library is already synchronized with the lockfile.
```

If you experience any trouble with this step, you might want to confirm that you are using R (v 4.5.2) in the RStudio IDE (v 2026.01.1+403) and Quarto (v 1.8.27). You can also read more about `renv()` in their [vignette](#)¹.

Ignored Files

Some project documents, scripts, and files are not distributed on GitHub or tracked by Git. Many of these are stored in `**/KEEP LOCAL/` directories or are explicitly listed under the “Project-Specific Exclusions” section of the `.gitignore` file.

Some of these files are available on the project SharePoint, while others are user-specific and do not need to be shared. If there are local-only files for this project, they will be added to the “Codebase Local Only Files” directory. This directory will contain a `README.rtf` file specifying the intended filepath for each file.

The YaleNew typeface must be downloaded directly by the user and requires NetID access: [Yale Typefaces](#). This typeface is only used by `SOP.qmd`. Place the downloaded files in

_styles/.

Content Curation

Policies

Curations are added directly to the “Pages/policies.qmd” file.

Courses and Education Programs Curation

The courses and educational programs curation is maintained in “Curated Educational Resources.xlsx”. These represent Yale University-wide courses, lecture materials, seminars, fellowships, workshops, symposia, or similar offerings.

1. Identify new candidate courses or educational programs not already listed in the “Courses & Events” sheet of “Curated Educational Resources.xlsx”. Candidates may also be submitted through the suggestion box Qualtrics survey or via direct contact.
2. Fill in all required fields (columns F through W) and any relevant optional or extended classifier fields (e.g., Institutional Home: “Other”). Required fields will be highlighted in red if left blank.
3. When adding content to the “Courses & Events” sheet of “Curated Educational Resources.xlsx”, initially set column V to “No”. This will highlight the row in blue, indicating that it needs to be added to the website.
4. Columns B through D are used to manage listings with expiration dates. Use columns B and C to indicate the expiration tracking method and timing, and column D to override expiration dating.

AI Tools Curation

The AI Tools tables are revised based on tools that are popular, frequently used, or listed in other Yale-affiliated curations. Curations are added directly to the “Pages/tools.qmd” file.

Committee Review of the Curation List

No procedure is defined.

Maintaining the Website

Page content is managed through the corresponding *.qmd files. Refer to existing files for guidance on content organization and formatting conventions.

Feedback and Suggestion Box

A user feedback survey (Microsoft Forms) and suggestion box (Qualtrics) have been created for this resource, with links available on the project SharePoint. Note that only designated collaborators have editing access to both. To edit the suggestion box, access it directly in Qualtrics rather than via the external link.

On the site, these are linked in three locations: a pop-up referenced in the *.qmd header YAML section, the main menu bar, and the right corner of the footer.

a. Configuring Stable Links

1. Launch the project by opening the *.Rproj file in RStudio.
2. Both stable links are accessible through the site's _quarto.yml. Open this *.yml file in RStudio.
3. Edit the main menu links under the navbar: section and the footer links under the right: subsection of page-footer: and save.
4. **OPTIONAL:** Preview the changes using a local preview. Navigate to the codebase directory in the command-line application (Terminal for Mac or Git Bash for Windows).

Command-Line Interface

```
cd "FILE-PATH/ai-resources-page/"
```

6. Run a local preview of the website. It will open in a browser at the "localhost:[port]/web/" address.

Command-Line Interface

```
quarto preview
```

7. To reflect changes saved in the *.yml file in the local preview, save a minor change in any *.qmd file to refresh the page. Be sure to revert these changes once the *.yml has been updated.

b. Configuring Pop-Up Links

The pop-up is coordinated using "Code/popup.js" and "_includes/feedback_popup.html", a before head file.

1. Launch the project by opening the *.Rproj file in RStudio.
2. Open "_includes/feedback_popup.html" to update the links and pop-up copy, then save the file.
3. **OPTIONAL:** Preview the changes using a local preview. Navigate to the codebase directory in the command-line application (Terminal for Mac or Git Bash for Windows).

Command-Line Interface

```
cd "FILE-PATH/ai-resources-page/"
```

4. Run a local preview of the website. It will open in a browser at the "localhost:[port]/web/" address.

Command-Line Interface

```
quarto preview
```

5. To reflect changes saved in the *.yml file in the local preview, save a minor change in a *.qmd file where the pop-up will appear to refresh the page. Be sure to revert these changes once the *.yml has been updated.

Timed Delay

The pop-up has a 2-minute delay. Allow the previewed page to sit for a few minutes before the pop-up appears.

Index and About Us

These pages do not require regular updates or maintenance. Links and copy can be revised directly in the respective HTML or Markdown sections. To update the People section of About Us, refer to the section on maintaining Listings.

Policies

As new sources of Yale or applicable AI-related policies are posted or released, links and copy can be revised directly in the respective HTML or Markdown sections.

AI Tools

The AI Tools page has two sections: links to frequently visited or other Yale-affiliated curation lists, and tables summarising popular AI tools used across different applications. Links and copy can be revised directly in the respective HTML or Markdown sections.

Listing Pages

Two pages utilize the Quarto Listing feature: the Courses and Educational Programs filter/search page and the People section of the About Us page. Both of these pages or sections is comprised of the following files:

- *.qmd — Renders the page on the website.
- *.yaml — Contains all information fields and organizes them within defined hierarchies and classifications.
- *.ejs — Ingests the *.yaml file and processes it through embedded HTML and JavaScript, formatting the tiles and defining page behaviors such as content-responsive text clamping and the “Show More” button.
- *.scss — Defines custom styling and some responsive behaviors.

The *.qmd files are located with in the “Pages/” directory, the *.yaml and *.ejs are in “Code/Listing_*/” respective directory, and the *.scss file is in the “_styles/” directory. Users only need to edit the *.yaml to remove, add, or update existing content fields.

Adding Tiles

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/ai-resources-page/"
```

2. Run a local preview of the website. It will open in a browser at the “localhost:[port]/web/” address.

Command-Line Interface

```
quarto preview
```

3. Launch the project by opening the *.Rproj file in RStudio.
4. Open the *.yaml and *.qmd files associated with the Listing you want to revise.
5. The header of each *.yaml file describes the required and optional fields used by the respective *.ejs file, as well as any additional hierarchies and classifications needed to organize the content. Refer to preexisting entries to help determine formatting and structural conventions.

Banner Color Scheme

The tile banner randomly generates a network graph each time the page is loaded. The colour schemes applied to this JavaScript are described in the “Progress Report” sheet of “Curated Educational Resources.xlsx”.

6. To refresh the local preview and reflect changes saved in the *.yaml, save a minor change in the associated *.qmd file. Be sure to revert these changes once you have finished updating the *.yaml.
7. Update the page-level “Date last updated: __” and “Last updated by: __” fields in the header section of the *.yaml file.
8. Update column V in the “Courses & Events” sheet of “Curated Educational Resources.xlsx” to “Yes”.
9. The content is now ready for publishing. This process is described below.

Removing Tiles

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/ai-resources-page/"
```

2. Run a local preview of the website. It will open in a browser at the "localhost:[port]/web/" address.

Command-Line Interface

```
quarto preview
```

3. Launch the project by opening the *.Rproj file in RStudio.
4. Open the *.yaml and *.qmd files associated with the Listing you want to revise.
5. Locate the title listing header for the tile to be removed. Delete all relevant components under this listing section and remove any extra spacing.
6. To refresh the local preview and reflect changes saved in the *.yaml, save a minor change in the associated *.qmd file. Be sure to revert these changes once you have finished updating the *.yaml.
7. Update the page-level "Date last updated: __" and "Last updated by: __" fields in the header section of the *.yaml file.
8. Update column V in the "Courses & Events" sheet of "Curated Educational Resources.xlsx" to "Removed".
9. The content is now ready for publishing. This process is described below.

Publishing Updates

A Shell script was created to render the site while preventing unused media from being transferred to the GitHub repository. Follow the steps below to render and publish the website.

1. Ensure all work is saved and committed.

Command-Line Interface

```
git add <"FILE-NAME" | .>
git commit -m "Your message"
git push
```

2. Render the site locally. The configuration updates the `_site` directory, which is excluded from version control by the `.gitignore` file. This process may take a few minutes to complete.

Command-Line Interface

```
quarto render
```

3. Make sure the shell script is executable.

Command-Line Interface

```
chmod +x Code/copy_images.sh
```

4. Run the script.

Command-Line Interface

```
./Code/copy_images.sh
```

5. **OPTIONAL:** Verify that all the correct files have been rendered to the `_site` directory.

Command-Line Interface

```
ls -R _site
```

6. If everything looks correct, publish the site. Follow the prompts by entering “Yes” to proceed with updating the site and entering any required passwords when prompted.

Command-Line Interface

```
quarto publish gh-pages --no-render
```

If Rendering Fails to Complete

The webpage may occasionally fail to publish; however, this is not necessarily a critical error. If this issue is encountered after attempting to publish, any working trees generated during the incomplete process must be cleared before re-executing the render. This is a known Quarto bug that may be resolved in a future update.

For example, if the working tree is called:

```
FILE-PATH/.quarto/quarto-publish-worktree-17fef8679f581e08/
```

1. List all currently active worktrees in the environment. One will be associated with `quarto publish` and denoted as “prunable.”

Command-Line Interface

```
git worktree list

# Example Result
FILE-PATH/ 69edfb0 [main]
FILE-PATH/.quarto/quarto-publish-worktree-71502552fdce11b5 0ee5bea
↪ [gh-pages] prunable
```

2. Forcably removed the prunable worktree.

Command-Line Interface

```
git worktree remove --force
↪ "FILE-PATH/.quarto/quarto-publish-worktree-71502552fdce11b5"
```

3. Retry publishing the site. Follow the prompts by entering “Yes” to proceed with updating the site and entering any required passwords when prompted.

Command-Line Interface

```
quarto publish gh-pages --no-render
```

Maintaining the SOP

This Standard Operating Procedure (SOP) uses a preconfigured LaTeX formatting script. The report *.qmd and formatting files are located in “Code/Standard Operating Procedure (SOP)”.

If you are updating the document, please add your name to the **Authors** field and update the **Date Updated** field accordingly. The LaTeX formatting script will automatically include the date the document was rendered at the top of the page.

Indented code blocks use Verbatim code environments, which require additional formatting to align the environment and apply code highlighting. Use “Code Highlighting Formatting_Verbatim.R” to prepare code for this environment and one-level indentation.

To preview a rendered version of these documents, comment out the following line in the website’s `_quarto.yml`:

`_quarto.yml`

```
- "!Code/Standard Operating Procedure (SOP)/SOP.qmd"
- "!Code/Standard Operating Procedure (SOP)/Code Highlighting
  ↪ Formatting_Verbatim.qmd"
```

This line prevents the SOP and its associated `*.qmd` files from being rendered when the site is published. Once editing and previewing are complete, uncomment the line to allow a new PDF to be rendered.

Once you have finished editing the document render it, move the resulting `*.pdf` to the project’s root directory, then commit and push your changes.

References

1. Ushey, K., Wickham, H. & Posit. [Introduction to renv](#).